But we do need to have an implementation, so let's pick a simple one. The following is a simple random number generator that uses the same algorithm as `scala.util.Random`, which happens to be what's called a *linear congruential generator* (http://mng.bz/r046). The details of this implementation aren't really important, but notice that `nextInt` returns both the generated value and a new `RNG` to use for generating the next value.

Listing 6.2 A purely functional random number generator

```
case class SimpleRNG(seed: Long) extends RNG {
  def nextInt: (Int, RNG) = {
    val newSeed = (seed * 0x5DEECE66DL + 0xBL) & 0xFFFFFFFFFFFFL
    val nextRNG = SimpleRNG(newSeed)
    val n = (newSeed >>> 16).toInt
    (n, nextRNG)
  }
}
```

The next state, which is an `RNG` instance created from the new seed.

`&` is bitwise AND. We use the current seed to generate a new seed.

`>>>` is right binary shift with zero fill. The value `n` is the new pseudo-random integer.

The return value is a tuple containing both a pseudo-random integer and the next `RNG` state.

Here's an example of using this API from the interpreter:

```
scala> val rng = SimpleRNG(42)
rng: SimpleRNG = SimpleRNG(42)

scala> val (n1, rng2) = rng.nextInt
n1: Int = 16159453
rng2: RNG = SimpleRNG(1059025964525)

scala> val (n2, rng3) = rng2.nextInt
n2: Int = -1281479697
rng3: RNG = SimpleRNG(197491923327988)
```

Let's choose an arbitrary seed value, 42.

Syntax for declaring two values by deconstructing the pair returned by `rng.nextInt`.

We can run this sequence of statements as many times as we want and we'll always get the same values. When we call `rng.nextInt`, it will always return `16159453` and a new `RNG`, whose `nextInt` will always return `-1281479697`. In other words, this API is pure.

6.3 Making stateful APIs pure

This problem of making seemingly stateful APIs pure and its solution (having the API *compute* the next state rather than actually mutate anything) aren't unique to random number generation. It comes up frequently, and we can always deal with it in this same way.[4]

[4] An efficiency loss comes with computing next states using pure functions, because it means we can't actually mutate the data in place. (Here, it's not really a problem since the state is just a single `Long` that must be copied.) This loss of efficiency can be mitigated by using efficient purely functional data structures. It's also possible in some cases to mutate the data in place without breaking referential transparency, which we'll talk about in part 4.

For instance, suppose you have a class like this:

```
class Foo {
  private var s: FooState = ...
  def bar: Bar
  def baz: Int
}
```

Suppose `bar` and `baz` each mutate `s` in some way. We can mechanically translate this to the purely functional API by making explicit the transition from one state to the next:

```
trait Foo {
  def bar: (Bar, Foo)
  def baz: (Int, Foo)
}
```

Whenever we use this pattern, we make the caller responsible for passing the computed next state through the rest of the program. For the pure `RNG` interface just shown, if we reuse a previous `RNG`, it will always generate the same value it generated before. For instance:

```
def randomPair(rng: RNG): (Int,Int) = {
  val (i1,_) = rng.nextInt
  val (i2,_) = rng.nextInt
  (i1,i2)
}
```

Here `i1` and `i2` will be the same! If we want to generate two distinct numbers, we need to use the `RNG` returned by the first call to `nextInt` to generate the second `Int`:

```
def randomPair(rng: RNG): ((Int,Int), RNG) = {
  val (i1,rng2) = rng.nextInt
  val (i2,rng3) = rng2.nextInt
   ((i1,i2), rng3)
}
```

Note use of `rng2` here.

We return the final state after generating the two random numbers. This lets the caller generate more random values using the new state.

You can see the general pattern, and perhaps you can also see how it might get tedious to use this API directly. Let's write a few functions to generate random values and see if we notice any repetition that we can factor out.

EXERCISE 6.1

Write a function that uses `RNG.nextInt` to generate a random integer between `0` and `Int.maxValue` (inclusive). Make sure to handle the corner case when `nextInt` returns `Int.MinValue`, which doesn't have a non-negative counterpart.

```
def nonNegativeInt(rng: RNG): (Int, RNG)
```

Dealing with awkwardness in functional programming

As you write more functional programs, you'll sometimes encounter situations like this where the functional way of expressing a program feels awkward or tedious. Does this imply that purity is the equivalent of trying to write an entire novel without using the letter *E*? Of course not. Awkwardness like this is almost always a sign of some missing abstraction waiting to be discovered.

When you encounter these situations, we encourage you to plow ahead and look for common patterns that you can factor out. Most likely, this is a problem that others have encountered, and you may even rediscover the "standard" solution yourself. Even if you get stuck, struggling to puzzle out a clean solution yourself will help you to better understand what solutions others have discovered to deal with similar problems.

With practice, experience, and more familiarity with the idioms contained in this book, expressing a program functionally will become effortless and natural. Of course, good design is still hard, but programming using pure functions greatly simplifies the design space.

EXERCISE 6.2

Write a function to generate a `Double` between `0` and `1`, not including `1`. Note: You can use `Int.MaxValue` to obtain the maximum positive integer value, and you can use `x.toDouble` to convert an `x: Int` to a `Double`.

```
def double(rng: RNG): (Double, RNG)
```

EXERCISE 6.3

Write functions to generate an `(Int, Double)` pair, a `(Double, Int)` pair, and a `(Double, Double, Double)` 3-tuple. You should be able to reuse the functions you've already written.

```
def intDouble(rng: RNG): ((Int,Double), RNG)
def doubleInt(rng: RNG): ((Double,Int), RNG)
def double3(rng: RNG): ((Double,Double,Double), RNG)
```

EXERCISE 6.4

Write a function to generate a list of random integers.

```
def ints(count: Int)(rng: RNG): (List[Int], RNG)
```